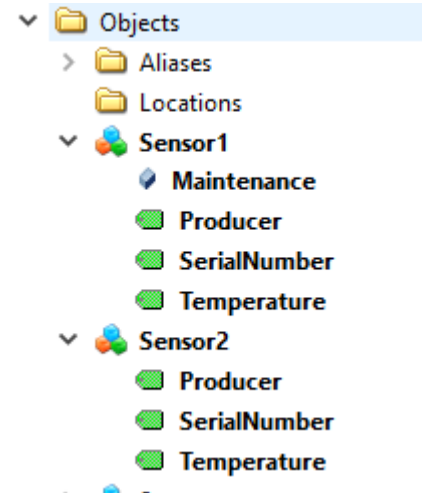


Esportare AddressSpace in UaModeler

- Provare a creare due oggetti di questo tipo:
- Verranno inseriti sempre nel namespace di indice 1
 - ns= 1 ; id=5003
 - ns= 1 ; id=5006



NodeId: Numeric 5003

DisplayName: Sensor1

BrowseName: 1 Sensor1

Description:

EventNotifier: 0 (None)

Children:

NodeId: Numeric 5006

DisplayName: Sensor2

BrowseName: 1 Sensor2

Description:

EventNotifier: 0 (None)

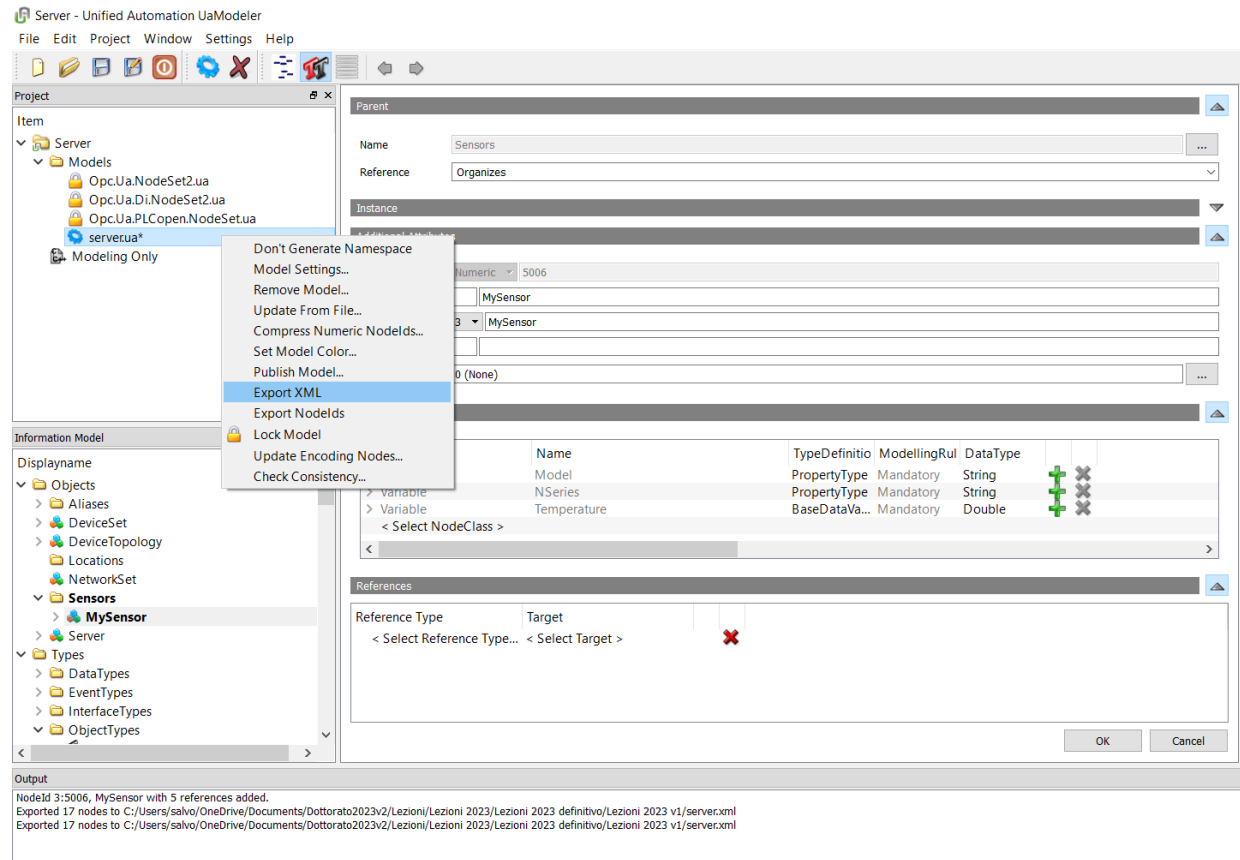
Children:

NodeClass	Name	TypeDefinition	ModellingRule	DataType		
> Variable	Producer	BaseDataVariableType	Mandatory	String	+	x
> Variable	SerialNumber	BaseDataVariableType	Mandatory	String	+	x
> Variable	Temperature	BaseDataVariableType	Mandatory	Double	+	x

NodeClass	Name	TypeDefinition	ModellingRule	DataType		
> Variable	Maintenance	PropertyType	Optional	String	+	x
> Variable	Producer	BaseDataVariableType	Mandatory	String	+	x
> Variable	SerialNumber	BaseDataVariableType	Mandatory	String	+	x
> Variable	Temperature	BaseDataVariableType	Mandatory	Double	+	x

Esportare AddressSpace in UaModeler

- Esportare in xml il nostro **NameSpace** personalizzato.



- Copiare il file xml all'interno della cartella in cui si desidera sviluppare il server in NodeJs

Struttura del Server

- Il programma che realizza un Server OPC UA dovrà essere articolato nella seguente maniera:
 1. Dichiarazioni varie
 2. Creazione dell'Istanza del server
 3. Inizializzazione del server
 4. Personalizzazione dell'AddressSpace
 5. Avvio del Server
- **Il programma è basato sul costrutto async/await**
 - Ci sono delle operazioni che devono essere eseguite in modo sincrono/bloccante
 - Inizializzazione del Server
 - Avvio del Server

Dichiarazioni

- Lo stack node-opcua è reso disponibile all'applicazione dall'istruzione 'require':

```
const opcua = require("node-opcua");
```

- Inseriamo nella variabile aspace il percorso del file xml esportato in precedenza

```
const aspace="./myserver_XML.xml"
```

- Nota: il file `myserver_XML.xml` è quello che trovate nella cartella su Teams

Dichiarazioni

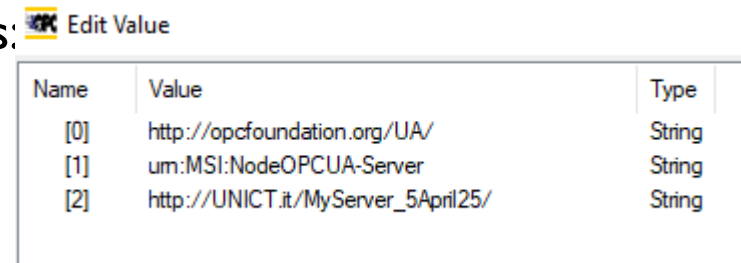
- Creiamo una variabile che contiene l'information model standard (OPC UA) più il custom namespace creato prima in UaModeler.

```
var xmlFiles = [  
    opcua.nodesets.standard,  
    aspace  
];
```

- Ricordarsi che l'AddressSpace avrà sempre i seguenti namespaces:

- Namespace di indice 0: contiene tutte le definizioni standard di OPC UA
- Namespace di indice 1: riservato al server che espone l'AddressSpace
- Namespace di indice 2,: usati nel caso di definizioni custom (come quella che abbiamo definito prima)

- **Questo significa che le nostre definizioni custom avranno indice 2 e non 1, come in ambito UA Modeller !!!!!**



The screenshot shows a window titled 'Edit Value' with a table containing three rows of namespace information. The columns are 'Name', 'Value', and 'Type'.

Name	Value	Type
[0]	http://opcfoundation.org/UA/	String
[1]	urn:MSI:NodeOPCUA-Server	String
[2]	http://UNICT.it/MyServer_5April25/	String

Istanza del server

- È necessario creare un'istanza del Server OPC UA. Per personalizzare il nostro server possono essere aggiunte delle opzioni che modificano il comportamento del server. La configurazione minima è:

```
const server = new opcua.OPCUAServer({  
  port: 4334,  
  resourcePath: "/UA/MyLittleServer",  
  nodeset_filename: xmlFiles,  
});
```

- La configurazione della porta e del resourcePath permetterà di costruire l'URI del

Discovery Endpoint del nostro server:

- `opc.tcp://<hostname>:4334/UA/MyLittleServer`
- `<hostname>` sarà sostituito dal nome del computer o dal nome di dominio completo.

Server started at
`opc.tcp://MSI:4334/UA/MyLittleServer`

- Al parametro `nodeset_filename` viene passata la variabile contenente l'AddressSpace definito precedentemente
- **importantissimo altrimenti che cosa espone il Server ?**

Inizializzazione del server

- Una volta creato il server deve essere inizializzato. Durante l'inizializzazione, il server caricherà il suo set di nodi predefinito e preparerà l'associazione di tutte le variabili OPCUA standard.
- L'operazione di inizializzazione **deve essere bloccante**, in quanto nessuna altra operazione può essere fatta fino a quando il server non viene inizializzato. Per questo motivo usiamo:

```
await server.initialize();
```

Personalizzazione dell'AddressSpace

- Una volta che il server è stato inizializzato, possiamo associare dei valori runtime ai nostri nodi personalizzati.
 - Ad esempio vogliamo assegnare dei valori agli oggetti dell'AddressSpace
- Per fare questo dobbiamo seguire i seguenti passaggi:
 1. Accedere all'addressSpace :

```
const addressSpace = server.engine.addressSpace;
```

2. Cercare attraverso il nodeId l'oggetto che contiene le proprietà e le variabili che vogliamo aggiornare.

```
const mySensor2=addressSpace.findNode('ns=2;i=5006')
```

I valori degli **indici i**, possono essere ricavati da UaModeler, per il namespace bisogna ricordarsi che se in UaModeler ns=1, allora Adesso il valore ns reale è 2

Personalizzazione dell'AddressSpace

3. Aggiornare il valore del componente dell'istanza. Ad esempio:

```
mySensor2.producer.setValueFromSource ({  
    dataType: opcua.DataType.String,  
    value: "Siemens"  
});
```

richiama ripetutamente una funzione o esegue un frammento di codice, con un ritardo fisso tra ogni chiamata.

```
setInterval(() => {  
    const value = 19 + 5 * Math.sin(Date.now() / 10000) + Math.random()*0.2;  
    mySensor2.temperature.setValueFromSource({  
        dataType: opcua.DataType.Double,  
        value  
    });  
}, 100);
```

100 ms

Avviare il Server

- Una volta che il server è stato creato e inizializzato, utilizziamo il metodo **start** per consentire al server di avviare tutti i suoi endpoint e iniziare ad ascoltare i client.
- Anche questa operazione è bloccante:

```
await server.start();  
console.log("server started at ", server.getEndpointUrl());
```

Server started at opc.tcp://MSI:4334/UA/MyLittleServer

Avviare il Server

- Il Server node-opcua espone solo un tipo di stack per il momento: **tcp-uasc-binary**
- Inoltre espone i seguenti Session EndPoints:

